Lab 1 — Pipes, dup2 and FIFOs

Try-at-home lab. Run it, break it, fix it, then build your own pipeline in C.

Operating Systems • Inha University in Tashkent • Fall 2026

COVERS	Slide 6 (*Pipes and dup2*) and Slide 7 (*FIFO and Pipeline*)
STATUS	Self-study practice. Nothing to submit. **The midterm doesn't assume you did it.**
YOU NEED	Your Ubuntu VM, a terminal, `gcc`. Some tasks need **two terminal windows** side by side.

Use only what the lectures gave you

Everything here uses `pipe`, `fork`, `close`, `read`, `write`, `dup2`, `execlp`, `wait` / `waitpid`, `sigaction`, `mkfifo`, `open` and `perror`. The few extra names you need are explained in grey boxes like this one, right where they first appear:

> **Compile** every program with `gcc -Wall -o name name.c` and run it with `./name`. `-Wall` turns on all the warnings — read them, they are usually right.

Part A — Try this at home

For every program: **type or paste it, compile it, run it, look at the output**, then answer the question in the blue box. Do not skip the "break it" steps — they are the point.

A1. Hello, pipe ★

This is the parent/child program from the lecture, cleaned up. The child writes, the parent reads.

```c
/* a1_hello_pipe.c — the parent/child pipe from the lecture */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    char buf[80] = "Hello from your child";
    char buf2[80];
```

```
    if (pipe(p) == -1) { perror("pipe"); exit(EXIT_FAILURE); }
    pid_t pid = fork();
    if (pid == 0) {                          /* child: the writer */
        close(p[0]);                         /* close the unused read end */
        write(p[1], buf, strlen(buf) + 1);  /* +1 sends the '\0' too */
        close(p[1]);
        exit(0);
    }
    /* parent: the reader */
    printf("parent: read end = fd %d, write end = fd %d\n", p[0], p[1]);
    close(p[1]);                             /* close the unused write end */
    ssize_t n = read(p[0], buf2, sizeof(buf2)); //int is often “okay” too instead of ssize_t
    printf("parent read %zd bytes: \"%s\"\n", n, buf2);
    close(p[0]);
    wait(NULL);
    return 0;
}
```

`%zd` is the `printf` format for an `ssize_t` (the "signed size" type that `read` and `write` return). **`wait(NULL)`**: the parent reaps the child, exactly as in Week 1.

```
$ gcc -Wall -o a1 a1_hello_pipe.c
$ ./a1
```

A2. Break the Sacred Rule of Closing ★

Now the child sends **three** messages, one per second, and the parent reads **in a loop until EOF** (the loop from `reader.c` on Slide 7).

```
/* a2_forgot_close.c — the child sends three messages; the parent reads until EOF */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    pipe(p);

    if (fork() == 0) {                       /* child: writer */
        close(p[0]);
        char *msgs[3] = { "one\n", "two\n", "three\n" };
        for (int i = 0; i < 3; i++) {
            write(p[1], msgs[i], strlen(msgs[i]));
            sleep(1);
        }
        close(p[1]);
        exit(0);
    }
```

```
    /* parent: reader */
    close(p[1]);          /* <-- STEP 2: put // in front of this line and run again */

    char buf[64];
    ssize_t n;
    while ((n = read(p[0], buf, sizeof buf - 1)) > 0) {
        buf[n] = '\0';                  /* make it a C string */
        printf("parent got %zd bytes: %s", n, buf);
    }
    printf("parent: read returned %zd -> EOF, done\n", n);
    wait(NULL);
    return 0;
}
```

`sleep(1)` pauses the calling process for one second. `buf[n] = '\0'`: `read` gives you bytes, not a C string; we add the terminating zero ourselves so `printf("%s")` knows where to stop — and we read at most `sizeof buf - 1` bytes so there is room for it.

Step 1. Compile and run. You should see three messages and then the EOF line.

Step 2. Put `//` in front of the line marked `STEP 2` (the parent's `close(p[1])`), compile, run again. Wait 10 seconds. Then press `Ctrl+C`.

QUESTION In step 2 the child has finished and exited. So who is still holding a write end of the pipe? Why does the kernel refuse to give the parent EOF? Why is there no error message — nothing crashes, nothing is logged?



A3. How big is a pipe? ★

One process, no `fork`. It creates a pipe and writes into it one byte at a time — and **nobody ever reads**.

```
/* a3_capacity.c — fill a pipe that nobody reads and see where write() stops */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    int p[2];
    pipe(p);
    /* We keep BOTH ends open and never read. One process, no fork. */
    char c = 'x';
    long total = 0;
```

```
    while (1) {
        write(p[1], &c, 1);
        total++;
        if (total % 4096 == 0)
            fprintf(stderr, "written so far: %ld bytes\n", total);
    }
    return 0;  /* never reached */
}
```

`fprintf(stderr, …)` is `printf` aimed at standard error (fd 2). We use it here because stderr is **not buffered**: each line appears the moment it is printed, even if the program then gets stuck.

```
$ ./a3

   ... watch the numbers, then press Ctrl+C
```

QUESTION What is the last number printed? Compare it with the number on the slide *pipe(int fd[2]): One Call, Two Ends*. The program has not crashed and has not exited — so where exactly is it, and what is it waiting for? Could anything ever wake it up?

A4. SIGPIPE — writing to nobody ★★

The read end is closed *before* the write. Nobody can ever read these bytes.

```
/* a4_sigpipe.c — write into a pipe whose read end is already closed */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    int p[2];
    pipe(p);
    close(p[0]);                        /* nobody can ever read now */

    write(STDOUT_FILENO, "about to write...\n", 18);
    write(p[1], "hi", 2);               /* the kernel sends SIGPIPE here */
    write(STDOUT_FILENO, "write finished\n", 15);   /* do we ever see this? */
    return 0;
}
```

```
$ ./a4

$ echo $?
```

Exit status 128 + N. When a process is **killed by a signal**, the shell reports its exit status as 128 plus the signal's number. Run `kill -l` to see the numbers; `SIGPIPE` is signal 13. Why it doesn't return 0?

Now catch the signal with `sigaction` — the Week 1 way:

```
#include <stdio.h> /* a4b_sigpipe_handler.c — same, but we catch SIGPIPE with sigaction (Week 1) */
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>
```

```
void on_pipe(int sig)
{
    char msg[] = "handler: caught SIGPIPE, nobody is reading!\n";
    write(STDERR_FILENO, msg, strlen(msg));
}

int main(void)
{
    struct sigaction sa = {0}; // just easier way to start
    sa.sa_handler = on_pipe;
    sigaction(SIGPIPE, &sa, NULL);

    int p[2];
    pipe(p);
    close(p[0]);

    ssize_t n = write(p[1], "hi", 2);
    printf("write returned %zd\n", n);
    if (n == -1) perror("write");
    return 0;
}
```

`errno = EPIPE`: if `SIGPIPE` does not kill the process, `write` returns `-1` and sets `errno` to `EPIPE`. `perror` turns that into the words *Broken pipe*.

QUESTION In `a4`, which line is the last one you see, and what does `echo $?` print? In `a4b`, the program survives: what does `write` return? You are writing a web server whose client just closed the browser tab. Do you want the default `SIGPIPE` behaviour or the handler? Why?

A5. The line that prints twice ★★

Remember from Slides: *printf is buffered; write goes to the kernel.* Here is where that bites.

```
/* a5_printf_twice.c — printf's buffer is copied by fork() */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("before fork\n");      /* goes into printf's buffer ... or does it? */
    pid_t pid = fork();
    if (pid == 0) {
        printf("I am the child\n");
        return 0;
    }
    wait(NULL);
    printf("I am the parent\n");
    return 0;
}
```

```
$ ./a5            # output to the terminal
$ ./a5 | cat      # output into a pipe
```

How printf decides when to flush. When stdout is a **terminal**, printf sends its buffer to the kernel at every `'\n'` (line buffered). When stdout is a **pipe or a file**, it waits until the buffer (several KB) is full or the program exits (fully buffered).

QUESTION How many times does `before fork` appear in each run? Explain the second run using the words *buffer*, *fork copies* and *exit*. How would you fix the program so it prints once in both cases? (Two fixes are possible with what you know.)

A6. Two directions = two pipes ★

The parent asks a question on one pipe, the child answers on the other.

```c
/* a6_two_pipes.c — two directions need two pipes */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int down[2];   /* parent -> child */
    int up[2];     /* child  -> parent */    pipe(down);       pipe(up);

    if (fork() == 0) {                          /* child */
        close(down[1]);                         /* child only reads "down" */
        close(up[0]);                           /* child only writes "up"  */
        char q[64];
        ssize_t n = read(down[0], q, sizeof q - 1);
        q[n] = '\0';
        printf("child : got question \"%s\"\n", q);
        char *a = "42";
        write(up[1], a, strlen(a));
        close(down[0]);
        close(up[1]);
        exit(0);
    }. /* parent */
    close(down[0]);         close(up[1]);
    char *q = "meaning of life?";
    write(down[1], q, strlen(q));
    char a[64];
    ssize_t n = read(up[0], a, sizeof a - 1);
    a[n] = '\0';
    printf("parent: got answer   \"%s\"\n", a);
    close(down[1]);
    close(up[0]);
    wait(NULL);
    return 0;
}
```

QUESTION Change the **parent** so that it `read`s from `up[0]` *before* it writes the question. Run it. What happens, and why? Then give it a name — it is the word from the *fork* slide.

A7. The child becomes ls — and never knows ★★

The dup2 → close → exec pattern from the slide
exec Keeps the Descriptors, now with the parent on the other end, counting what `ls` wrote.

```c
/* a7_ls_to_parent.c — the child becomes ls, the parent reads what ls printed */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    pipe(p);

    if (fork() == 0) {                /* child: will become ls */
        dup2(p[1], 1);                /* 1. stdout now goes into the pipe */
        close(p[0]);                  /* 2. close both original ends      */
        close(p[1]);
        execlp("ls", "ls", "-l", "/", NULL);   /* 3. become ls */
        perror("execlp");             /* only reached if exec failed */
        exit(1);
    }

    /* parent: reads everything ls wrote, counts bytes and lines */
    close(p[1]);
    char buf[256];
    ssize_t n;
    long bytes = 0, lines = 0;
    while ((n = read(p[0], buf, sizeof buf)) > 0) {
        bytes += n;
        for (int i = 0; i < n; i++)
            if (buf[i] == '\n') lines++;
    }
    close(p[0]);
    wait(NULL);
    printf("ls wrote %ld bytes in %ld lines, and never knew it was talking to a pipe\n",
           bytes, lines);
    return 0;
}
```

```
$ ./a7

$ ls -l / | wc -lc      # the shell's answer, to compare
```

`wc -lc` counts **l**ines and **c**haracters (bytes) of its input.

QUESTION Do your numbers match the shell's? Now change `"ls"` in `execlp` to `"lss"` (a program that does not exist). What does the parent report, and where does the `perror` message appear — in the pipe or on your screen? Why?

A8. FIFOs — two terminals, two strangers ★

Step 1 — pure shell, no C. Open two terminals.

```
# terminal 1                          # terminal 2
$ mkfifo /tmp/myfifo
$ ls -l /tmp/myfifo
$ cat /tmp/myfifo                     $ echo "hello from bash" > /tmp/myfifo
```

> **`> file`** in the shell sends a command's standard output into `file` instead of the screen. (Part B shows you how the shell does this with `open` + `dup2`.)

Step 2 — the programs from Slide 7.

```c
/* writer.c — creates the FIFO and sends one message (from Slide 7) */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>

#define FIFO_PATH "/tmp/demo_fifo"

int main(void)
{
    if (mkfifo(FIFO_PATH, 0666) == -1 && errno != EEXIST) {
        perror("mkfifo");
        exit(EXIT_FAILURE);
    }
    printf("writer: waiting for a reader (open blocks)...\n");
    int fd = open(FIFO_PATH, O_WRONLY);          /* <- blocks here */
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }
    printf("writer: reader arrived, sending\n");
    char *msg = "hello over FIFO\n";
    if (write(fd, msg, strlen(msg)) == -1) perror("write");
    close(fd);
    return 0;
}
```

```c
/* reader.c — receives everything until the writer closes (from Slide 7) */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO_PATH "/tmp/demo_fifo"

int main(void)
{
    printf("reader: waiting for a writer (open blocks)...\n");
    int fd = open(FIFO_PATH, O_RDONLY);          /* <- blocks here */
    if (fd == -1) { perror("open"); exit(EXIT_FAILURE); }
    printf("reader: writer arrived\n");
    char buf[256];
    ssize_t n;
    while ((n = read(fd, buf, sizeof buf)) > 0)
        write(STDOUT_FILENO, buf, n);
    /* n == 0 -> all writers closed (EOF) */
    printf("reader: EOF, done\n");
    close(fd);
    return 0;
}
```

Try these orders, and write down what you see in each terminal:

1. Start `./writer` first, wait 5 s, then start `./reader`.
2. Start `./reader` first, wait 5 s, then `./writer`.
3. Run `rm /tmp/demo_fifo`, then start `./reader` first. Why is this different from order 2?
4. Start `./reader`, then in the other terminal: `echo one > /tmp/demo_fifo`.
   Does the reader stop after the first message?

QUESTION Which call does each program block in, and what exactly unblocks it? Check your answer against the `open()` table on slides. What does `ls -l` show as the **size** of the FIFO while data is flowing?

Part B — The missing half: `ls -l | wc -l` in C

In the lecture we built the **writer** side of a pipeline: a child that rewires its stdout into a pipe and becomes `ls`. A real pipeline also needs the **reader** side — a second child that rewires its **stdin** to the pipe and becomes `wc` — and a parent that behaves like the shell. Here is the whole thing.

The recipe: one pipe · two forks · each child dup2s its end and closes both originals · the parent closes both ends · two waitpids.

```c
/* b1_ls_wc.c — the shell's  ls -l | wc -l  written by hand */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    if (pipe(p) == -1) { perror("pipe"); exit(EXIT_FAILURE); }

    /* ---------- child 1: the WRITER side, becomes ls -l ---------- */
    pid_t left = fork();
    if (left == 0) {
        dup2(p[1], 1);            /* stdout (1) -> write end of the pipe */
        close(p[0]);              /* close #1: ls never reads the pipe   */
        close(p[1]);              /* close #2: fd 1 already points there */
        execlp("ls", "ls", "-l", NULL);
        perror("execlp ls");      /* only reached if exec failed */
        exit(1);
    }

    /* ---------- child 2: the READER side, becomes wc -l ---------- */
    pid_t right = fork();
    if (right == 0) {
        dup2(p[0], 0);            /* stdin (0) -> read end of the pipe   */
        close(p[0]);              /* close #3: fd 0 already points there */
        close(p[1]);              /* close #4: THE important one — if wc
                                     holds a write end, wc never sees EOF */
        execlp("wc", "wc", "-l", NULL);
        perror("execlp wc");
        exit(1);
    }

    /* ---------- parent: the shell. It uses NEITHER end. ---------- */
    close(p[0]);                  /* close #5 */
    close(p[1]);                  /* close #6: without it, wc waits forever */

    waitpid(left, NULL, 0);       /* reap both children — no zombies */
    waitpid(right, NULL, 0);
    printf("parent: both children finished\n");
    return 0;
}
```

```
$ ./b1

$ ls -l | wc -l          # must print the same number
```

What happens to the descriptor tables

Read this table one row at a time. `tty` means your terminal; `R` and `W` are the read and write ends of the pipe; a dash means "closed".

Moment	fd 0	fd 1	fd 3	fd 4
Parent, right after `pipe(p)`	tty	tty	R	W
Both children, right after `fork` (exact copies)	tty	tty	R	W
Child 1 after `dup2(p[1],1)`	tty	**W**	R	W
Child 1 after its two closes → becomes `ls`	tty	**W**	–	–
Child 2 after `dup2(p[0],0)`	**R**	tty	R	W
Child 2 after its two closes → becomes `wc`	**R**	tty	–	–
Parent after its two closes	tty	tty	–	–

Now count the **write ends** left in the whole system: exactly one — `ls`'s fd 1. When `ls` finishes and exits, that last write end closes, and `wc`'s next `read` returns 0. That is how `wc` knows it may print its answer.

Break it on purpose

Take the six `close` calls, labelled `#1` to `#6` in the comments. For each one, **first predict** what happens if you delete only that line, **then test it**. Fill in the table.

Delete only…	Your prediction (works / hangs)	What really happened
close #1 — `ls` child's read end		
close #2 — `ls` child's original write end		
close #3 — `wc` child's original read end		
close #4 — `wc` child's write end		
close #5 — parent's read end		
close #6 — parent's write end		

QUESTION Some deletions hang and some do not. Find the **one rule** that tells them apart.
(Hint: which of the three processes is the one waiting for EOF, and which ends does EOF depend on?)

B2. The shell's `>` and `<` are the same trick

You have used `echo … > /tmp/myfifo`. Here is what the shell does for `>`:
it opens the file and dup2s it onto fd 1, then execs the command. I will also teach this in class in detail

```c
/* b2_redirect_out.c — the shell's  ls -l > listing.txt  written by hand */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
```

```c
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        int fd = open("listing.txt",O_WRONLY|O_CREAT|O_TRUNC, 0644); // you don't have to memorize this
        if (fd == -1) { perror("open"); exit(1); }
        dup2(fd, 1);              /* stdout -> the file */
        close(fd);                /* fd 1 is enough */
        execlp("ls", "ls", "-l", NULL);
        perror("execlp");
        exit(1);
    }
    wait(NULL);
    printf("parent: done — now run  cat listing.txt\n");
    return 0;
}
```

`O_WRONLY | O_CREAT | O_TRUNC, 0644`: open for writing; **create** the file if it does not exist; **truncate** it (empty it) if it does. The `|` here is C's bitwise OR, which combines flags. `0644` is the permission of a newly created file: owner may read and write, everyone else may only read.

And `<` is the same idea on fd 0:

```c
/* b3_redirect_in.c — the shell's  wc -l < /etc/passwd  written by hand */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        int fd = open("/etc/passwd", O_RDONLY);
        if (fd == -1) { perror("open"); exit(1); }
        dup2(fd, 0);              /* stdin <- the file */
        close(fd);
        execlp("wc", "wc", "-l", NULL);
        perror("execlp");
        exit(1);
    }
    wait(NULL);
    return 0;
}
```

You type in the shell	What the shell does in the child before `exec`
`cmd > file`	`fd = open(file, O_WRONLY|O_CREAT|O_TRUNC, 0644); dup2(fd, 1); close(fd);`
`cmd < file`	`fd = open(file, O_RDONLY); dup2(fd, 0); close(fd);`
`a | b`	`pipe(p)`; in `a`: `dup2(p[1], 1)`; in `b`: `dup2(p[0], 0)`; everyone closes both originals

Part C — Your turn

No code given now — only the goal, the expected output, and hints.
Every task can be solved with what is in Parts A and B. Test each program against the real shell command.

C1. Feed a sorter ★★

Write `c1.c`. The **parent** writes these five lines into a pipe: `42`, `7`, `1000`, `-3`, `15`. The **child** becomes `sort -n` with its stdin coming from the pipe. This time the parent is the writer and the exec'd child is the reader — the mirror image of A7.

> **`sort -n`** reads all lines from stdin, then prints them sorted as **n**umbers. It cannot print anything until it has seen EOF — it has to know it has the last line.

```
$ ./c1
-3
7
15
42
1000
```

QUESTION Which `close` in the **parent** is the one that makes `sort` start printing?
What happens if you put it after `waitpid` instead of before?

C2. Three stages ★★

Write `c2.c` that does exactly what this shell line does:

```
$ cat /etc/passwd | grep sh | wc -l
```

> **`grep sh`** keeps only the lines that contain `sh`. **`/etc/passwd`** is the list of user accounts;
> the line ends with the user's login shell.

- You need **two pipes** and **three children**.
  The middle child (`grep`) uses **both**: stdin from the first pipe, stdout into the second.
- Create both pipes before the first `fork`.
  Then every child inherits **four** pipe descriptors and must close all four after its `dup2`s.
- The parent closes all four and waits for all three children.

QUESTION Count the `close` calls in your program.
Which ones, if forgotten, would make it hang? Which process would be stuck?

C3. Capture any command ★★

Write `capture.c`. It runs **whatever command you give it on the command line**, catches everything the command prints, and reports:

```
$ ./capture ls -l /
[capture] 'ls' printed 20 lines, 941 bytes
$ ./capture echo hello world
[capture] 'echo' printed 1 lines, 12 bytes
```

`execvp(file, argv)` is `execlp` with the arguments in an **array** instead of a list — useful when you do not know at compile time how many there are. The array must end with `NULL`. Your own `argv` already ends with `NULL`, so `execvp(argv[1], &argv[1])` runs "everything after the program name". **`argc`** is how many words were typed, including `./capture` itself.

QUESTION What should your program print for `./capture nosuchprogram`? Make sure it does not hang.

C4. A pipeline of any length ★★★

Write `pipeline.c`. Use `+` on the command line where the shell would use `|`:

```
$ ./pipeline ls -l /usr/bin + grep zip + wc -l
$ ls -l /usr/bin | grep zip | wc -l        # same number
$ ./pipeline cat /etc/passwd + grep sh + sort + head -2
$ ./pipeline echo just one stage
```

`strcmp(a, b)` (from `<string.h>`) returns 0 when two strings are equal. **`head -2`** prints only the first 2 lines of its input.

Hints — read only when stuck:

1. Walk through `argv`. Every time you meet `"+"`, replace it with `NULL` and remember that the next word starts a new stage. Now every stage is a ready-made, `NULL`-terminated array for `execvp` — no copying needed.
2. Do not create all the pipes first. Use one loop over the stages. Keep a variable `prev_read`: the read end of the pipe made in the previous turn of the loop (`-1` for the first stage).
3. Stage *s* child: if `prev_read != -1`, dup2 it to 0. If it is not the last stage, dup2 the new pipe's write end to 1. Close everything you dup2'd from.
4. The parent, after each `fork`, closes `prev_read` and the new write end, and keeps the new read end as the next `prev_read`. Store every pid; wait for all at the end.

QUESTION With this design, how many pipe descriptors does the parent hold open at any moment, no matter how long the pipeline is? Why does that matter for a pipeline of 1000 stages?

C5. A FIFO log server for many strangers ★★

Write `logger.c`: a server that creates `/tmp/soc3010_log` and prints every line any process writes into it, labelled with a client number. Clients are plain shell commands in other terminals:

```
# terminal 2                                    # terminal 1 (logger)
$ echo "disk almost full" > /tmp/soc3010_log      [client 1] disk almost full
                                                   logger: client 1 left
$ echo "backup done" > /tmp/soc3010_log           [client 2] backup done
                                                   logger: client 2 left
```

1. Start from `reader.c` (Slide 7). Run it with `FIFO_PATH` changed and one `echo`. It stops after the first client. **Why?**
2. Fix it so the server keeps serving client after client. (What should happen when `read` returns 0?)
3. When you stop the server with `Ctrl+C`, the FIFO stays in `/tmp`. Install a `SIGINT` handler with `sigaction` that removes it and exits.

> **`unlink(path)`** (from `<unistd.h>`) removes a name from the filesystem — what `rm` does. It was the comment at the end of `reader.c`. **`_exit(0)`** ends the process immediately; it is the version that is safe to call inside a signal handler.

This is many writers → one reader. Your FIFO homework is two programs taking turns — a different problem.

C6. Boss level: a conversation with another program ★★★★

Goal: your program sends text to `tr a-z A-Z` and reads the upper-cased text **back**. `tr` reads stdin and writes stdout, so you need **two pipes**: one into its stdin, one out of its stdout.

> **`tr a-z A-Z`** translates characters: every lowercase letter it reads is written back in uppercase.
> Try it in the shell: `echo hello | tr a-z A-Z`.

Step 1. Here is the obvious design. Run it. It hangs.
Explain exactly why — *which process is blocked in which call*. (Hint: A5.)

```c
/* c6_naive.c — the obvious design, which hangs */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int to_tr[2], from_tr[2];
    pipe(to_tr);
    pipe(from_tr);
```

```
    if (fork() == 0) {
        dup2(to_tr[0], 0);
        dup2(from_tr[1], 1);
        close(to_tr[0]); close(to_tr[1]);
        close(from_tr[0]); close(from_tr[1]);
        execlp("tr", "tr", "a-z", "A-Z", NULL);
        perror("execlp"); exit(1);
    }
    close(to_tr[0]);
    close(from_tr[1]);

    char *line = "hello\n";
    write(to_tr[1], line, strlen(line));     /* send one line ...              */
    char buf[64];
    ssize_t n = read(from_tr[0], buf, sizeof buf - 1);   /* ... wait for the answer */
    buf[n] = '\0';
    printf("answer: %s", buf);
    close(to_tr[1]);
    close(from_tr[0]);
    wait(NULL);
    return 0;
}
```

Step 2. Fix it for one short line, still with one child.

Step 3. Now send the line **20 000 times** (≈ 800 KB). Your step 2 fix hangs again. Draw both pipes, mark which one is full, and say who is waiting for whom.

Step 4. Fix it for any input size. You may use one extra `fork`. Print the first line you get back and the total number of bytes:

```
$ ./upper 20000

HELLO FROM THE OPERATING SYSTEMS COURSE

parent: got 800000 bytes back
```

Part D — Think it through (pen and paper)

Answer without running the code first. Then run it and check yourself.

D1. Draw the tables ★★

For your **C2** program (`cat | grep | wc`), draw the descriptor table of all **four** processes at the moment just before each child calls `execlp` (and the parent's after its closes). Use the same style as the table in Part B: fd 0, fd 1, fd 3, 4, 5, 6. How many **write ends** of pipe `a` exist in the system at that moment? Of pipe `b`?

D2. "Let it finish, then I'll read" ★★★

```c
/* d2_wait_first.c — the parent waits for the child BEFORE reading */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int p[2];
    pipe(p);
    pid_t pid = fork();
    if (pid == 0) {
        dup2(p[1], 1);
        close(p[0]); close(p[1]);
        execlp("seq", "seq", "1", "100000", NULL);   /* about 590 KB of output */
        perror("execlp"); exit(1);
    }
    close(p[1]);
    waitpid(pid, NULL, 0);          /* "let it finish first, then I read" */
    char buf[4096];
    ssize_t n;
    long total = 0;
    while ((n = read(p[0], buf, sizeof buf)) > 0) total += n;
    printf("read %ld bytes\n", total);
    return 0;
}
```

`seq 1 100000` prints the numbers 1 to 100 000, one per line — about 590 000 bytes.

(a) This program hangs forever. Explain why in two sentences.
(b) Would it still hang with `seq 1 1000` (about 3 900 bytes)?
(c) Roughly, what is the largest output for which this design works?
(d) Fix it by moving one line.

D3. Did I just close stdout? ★★

```c
dup2(p[1], 1);    close(p[1]);     printf("where does this go?\n");
```

After `close(p[1])`, is the pipe's write end closed for this process?
Where does the `printf` text go? Answer using the words *ticket number* and *open file description*.

D4. Wrong order ★★

```c
if (fork() == 0) {
    execlp("ls", "ls", NULL);
    dup2(p[1], 1);
    close(p[0]); close(p[1]);
}
```

Where does `ls`'s output go?
Will the `dup2` line ever run? If a parent is reading `p[0]` until EOF, will it ever finish?

D5. Messages without edges ★★

A child does three `write` calls very fast, with no `sleep`: `"one"`, `"two"`, `"three"` (no newlines, no `'\0'`). The parent then does a **single** `read(p[0], buf, 64)`. List two different things the parent might receive. Which statement on Slides (*Why not a shared file?*) does this remind you of — and do pipes solve that problem?

D6. Two FIFOs, one bad order ★★★

Programs X and Y talk over two FIFOs: X → Y on `/tmp/f1`, Y → X on `/tmp/f2`. Each program first opens the FIFO it **reads**, then the one it **writes**:

```
/* X */                                  /* Y */
int in  = open("/tmp/f2", O_RDONLY);     int in  = open("/tmp/f1", O_RDONLY);
int out = open("/tmp/f1", O_WRONLY);     int out = open("/tmp/f2", O_WRONLY);
```

Both FIFOs already exist. What happens when you start X and Y? Name the call each one is stuck in, using the `open()` table from Slide 7. Then fix it by changing **one** program only.

D7. Where do the bytes go? ★★

In A7, a student forgets **only** the `dup2(p[1], 1)` line in the child. What appears on the screen? What does the parent report? Does the program finish?

D8. Count ★

For the shell command `a | b | c | d`: how many processes does the shell fork? How many times does it call `pipe()`? How many descriptors does `pipe()` create in total? How many `waitpid` calls should the shell make?

D9. True or false — and one sentence why ★

a. `read` on a pipe returns 0 when the pipe is empty right now.
b. After `fork`, the pipe has two buffers — one for the parent, one for the child.
c. `exec` closes all open file descriptors.
d. `dup2(3, 1)` closes fd 3.
e. Data written to a FIFO is stored in the file on disk until someone reads it.
f. `write` into a full pipe returns an error.
g. In `ls | wc`, if the `ls` child forgets to close its copy of the **read** end, the pipeline still works.
h. If nobody handles it, `SIGPIPE` kills the process.
i. `open(fifo, O_RDONLY)` returns immediately if the FIFO is empty.